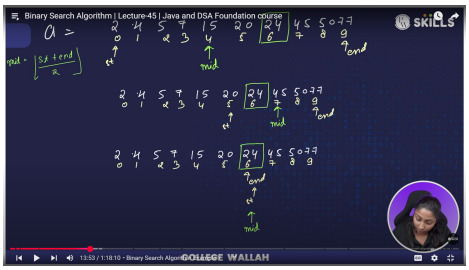
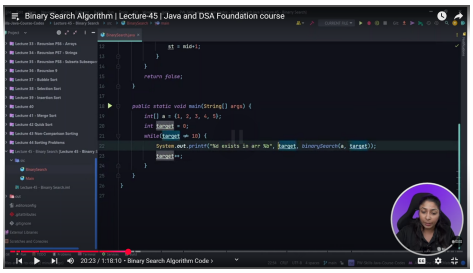
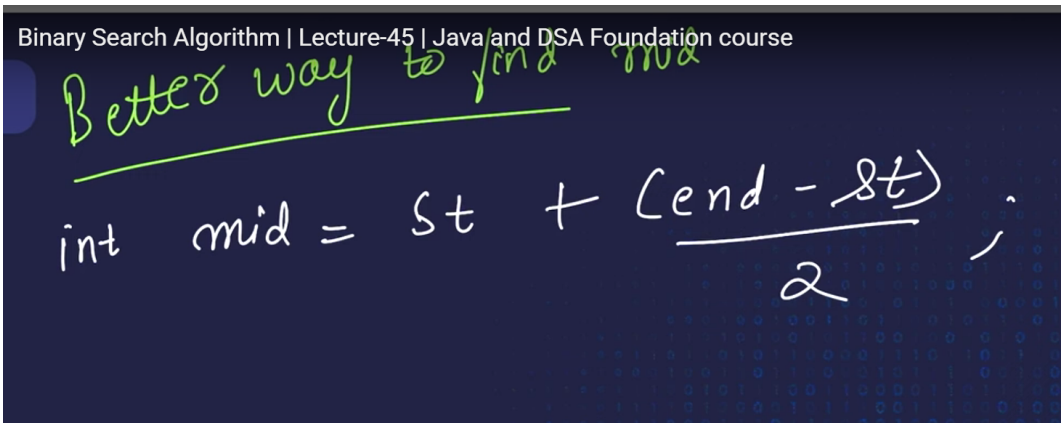
Binary Search

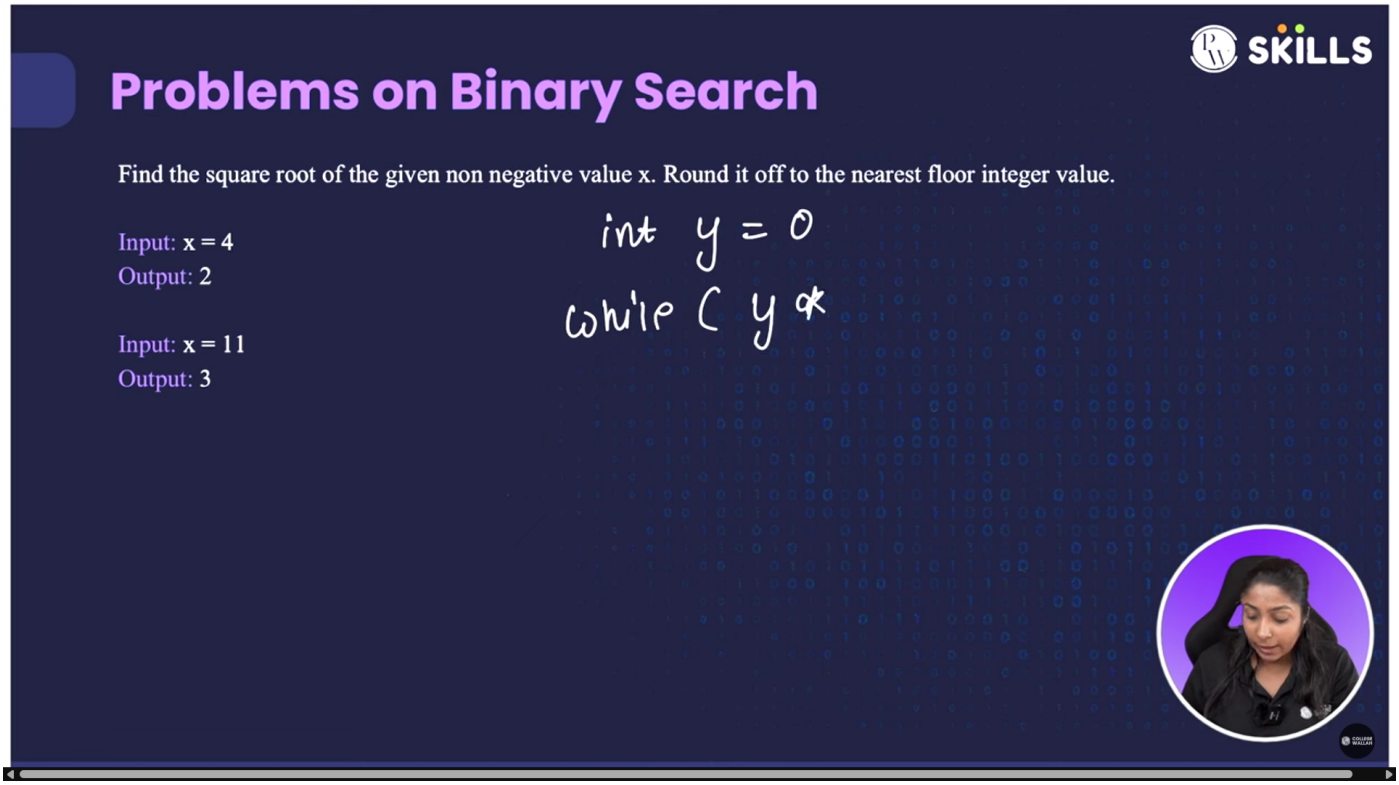
08 July 2025 17:13

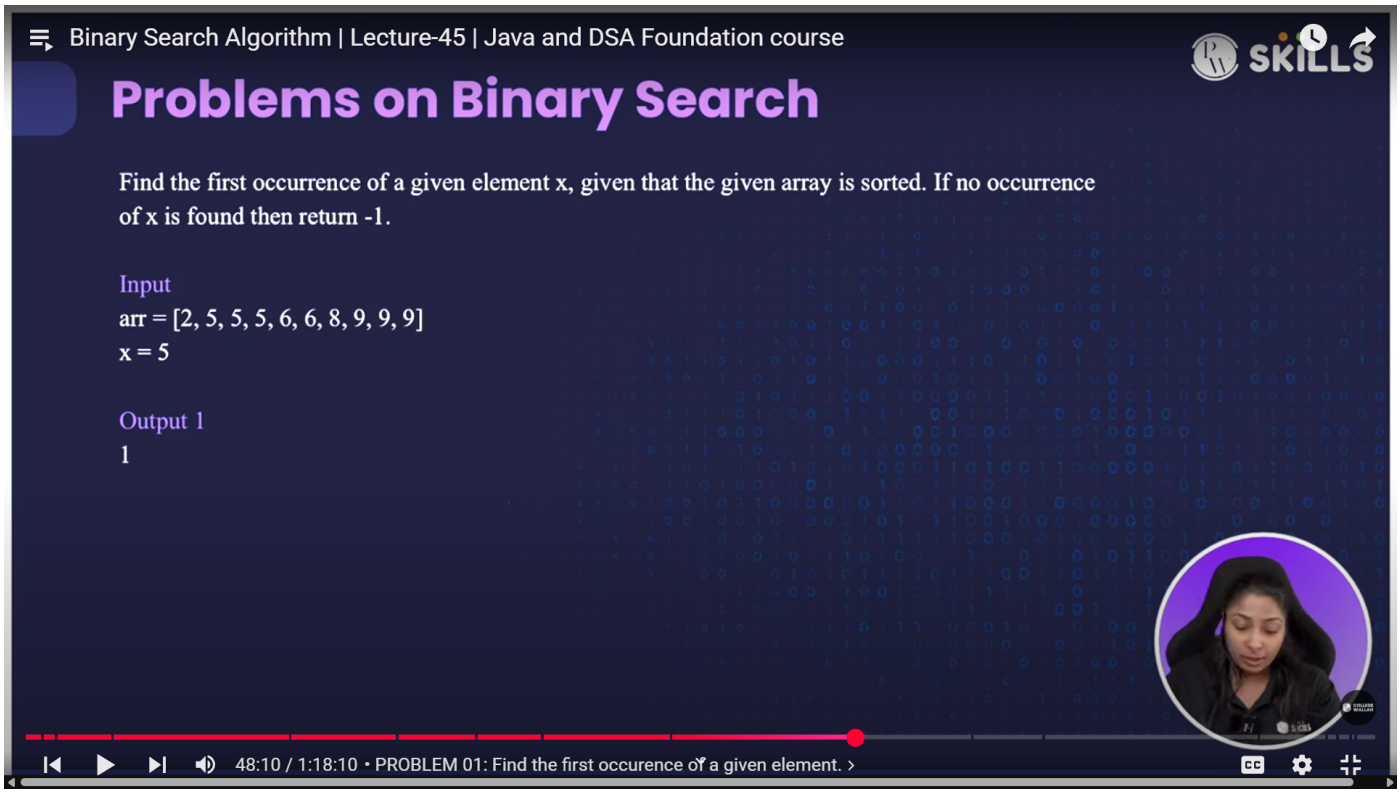


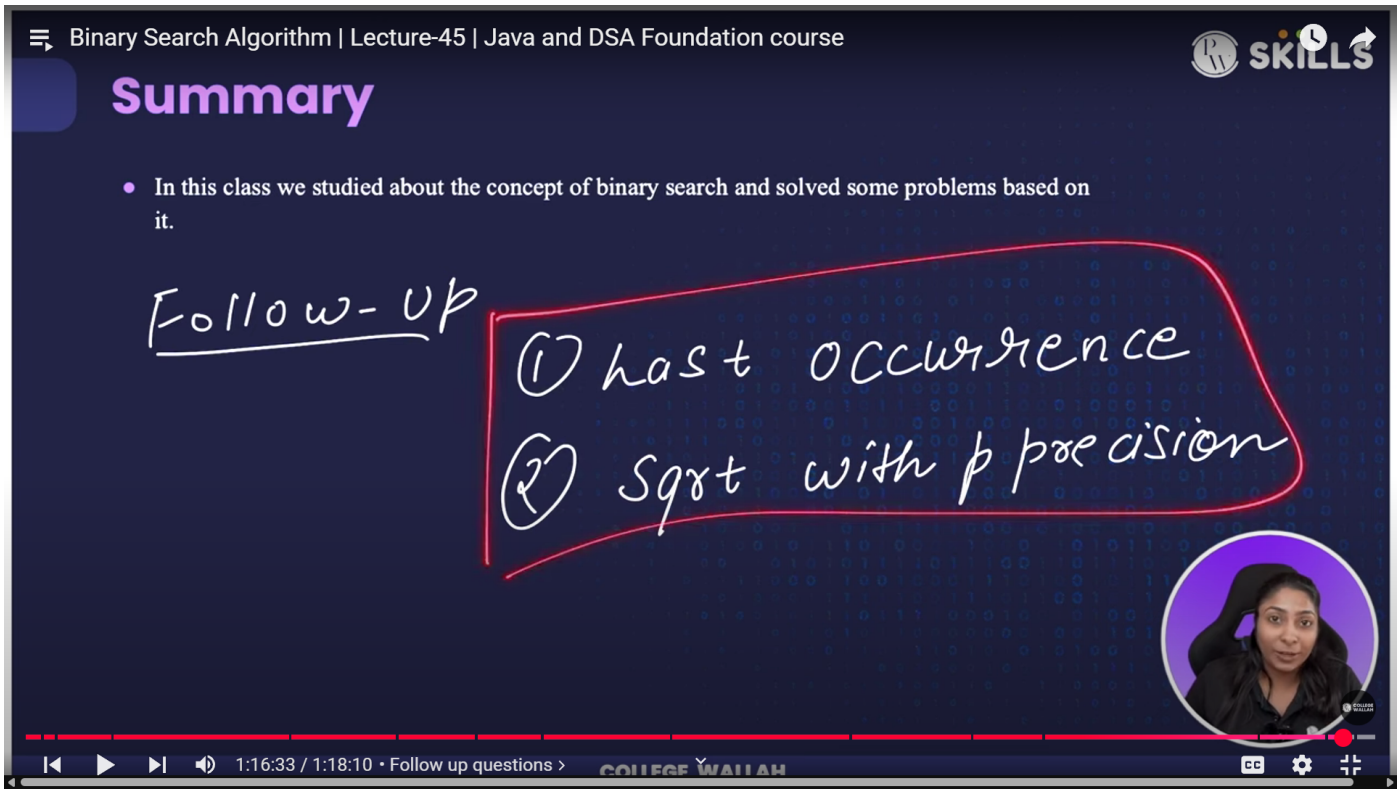


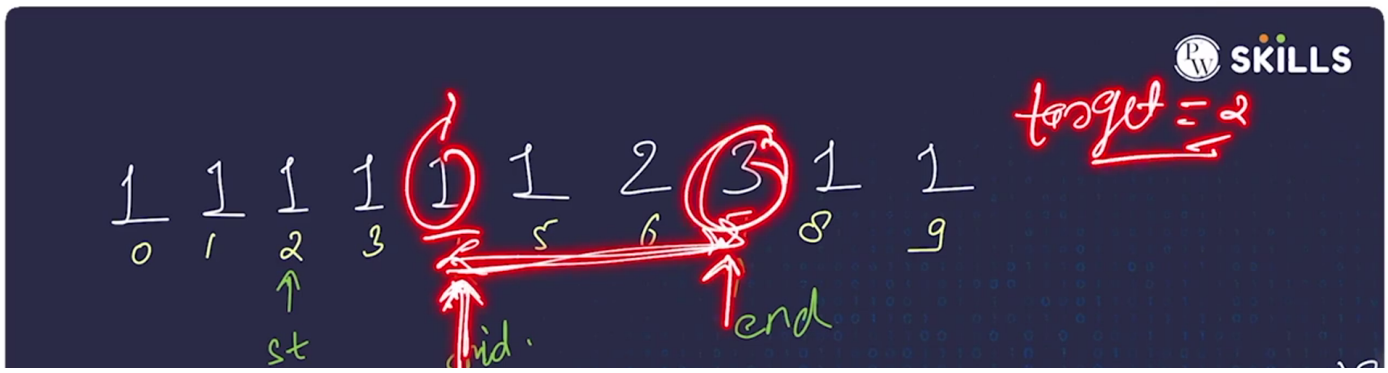


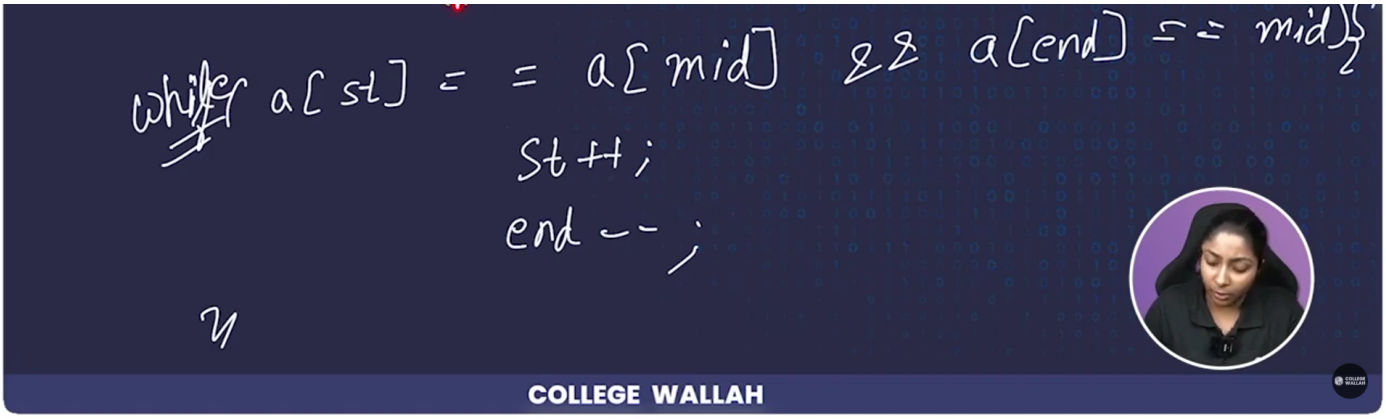
Time Complexity : O(logn) (For both iterativ e and recursive)
Space complexity of iterative -> O(1)
Space Complexity of recursive -> O(logn)



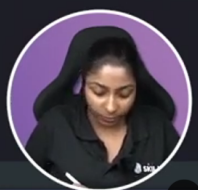

Binary Search Algorithm | Lecture-45 | Java and DSA Foundation course
SKILLS
Problems on Binary Search
Find the first occurrence of a given element x, given that the given array is sorted. If no occurrence of x is found then return -1.
Input
arr = [2, 5, 5, 5, 6, 6, 8, 9, 9, 9]
x = 5
Output 1
1
48:10 / 1:18:10 • PROBLEM 01: Find the first occurence of a given element. ›

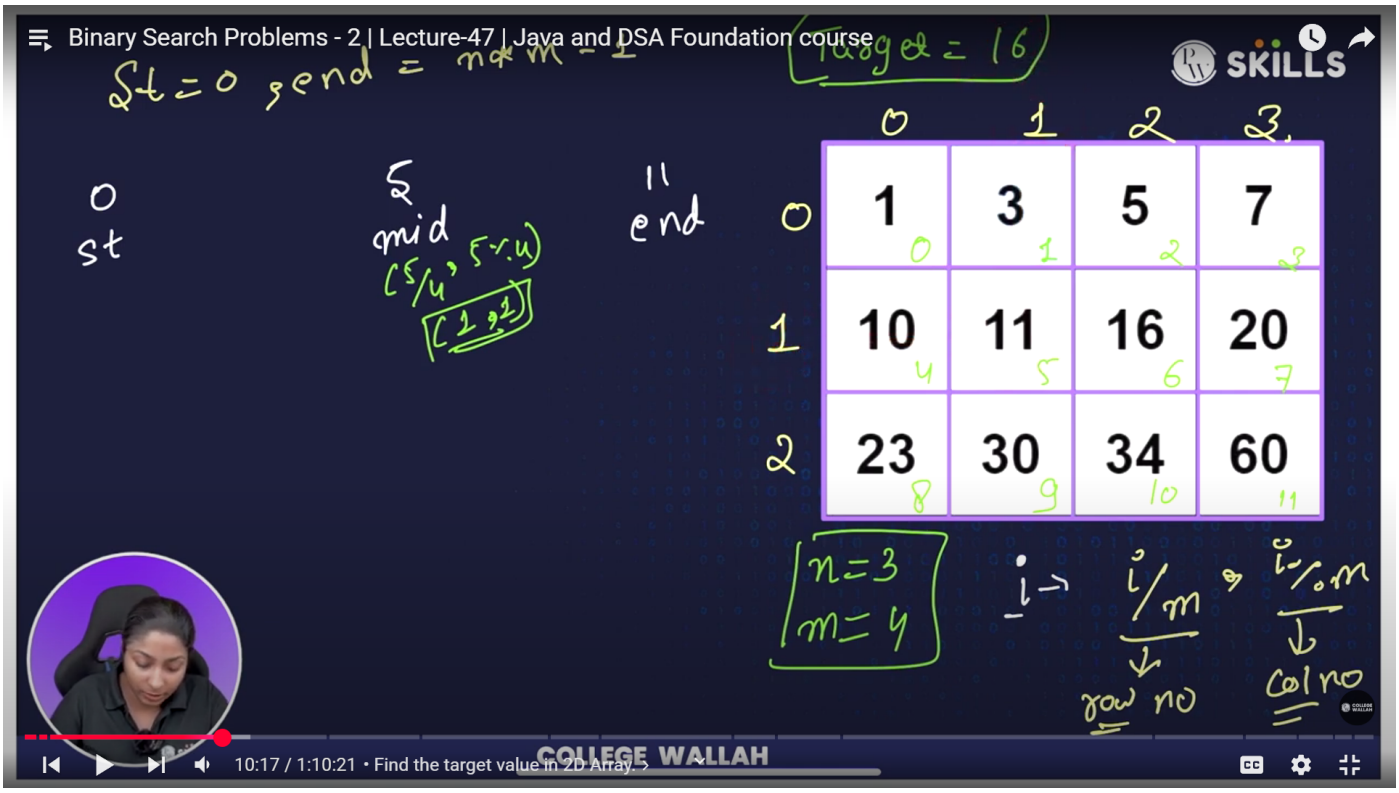

Binary Search Algorithm | Lecture-45 | Java and DSA Foundation course
SKILLS
Summary
In this class we studied about the concept of binary search and solved some problems based on it.
Follow-up
① last occurrence
② sqrt with p precision
1:16:33 / 1:18:10 • Follow up questions ›
COLLEGE WALLAH

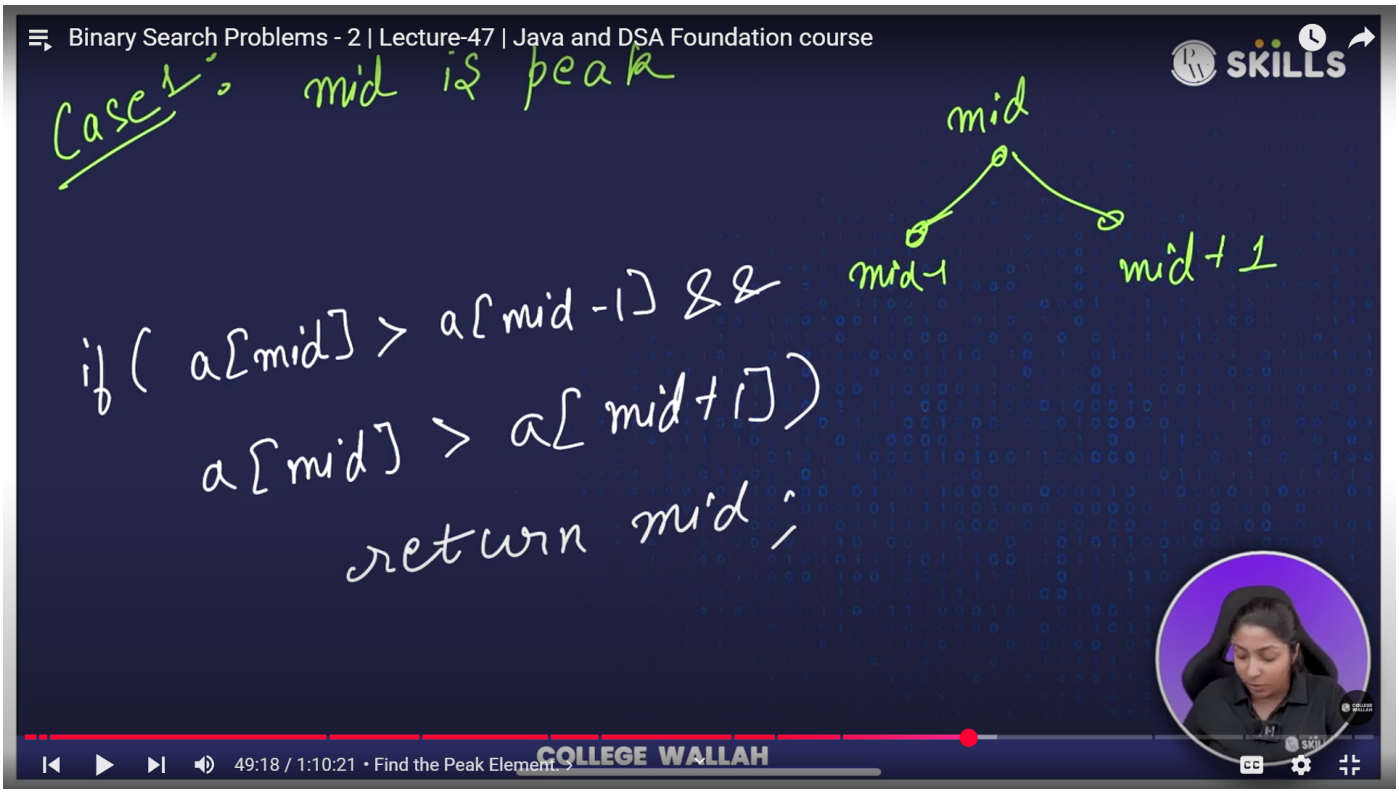

SKILLS
target = 2
1 1 1 1 1 1 2 3 1 1
0 1 2 3 5 6 8 9
st
mid
end

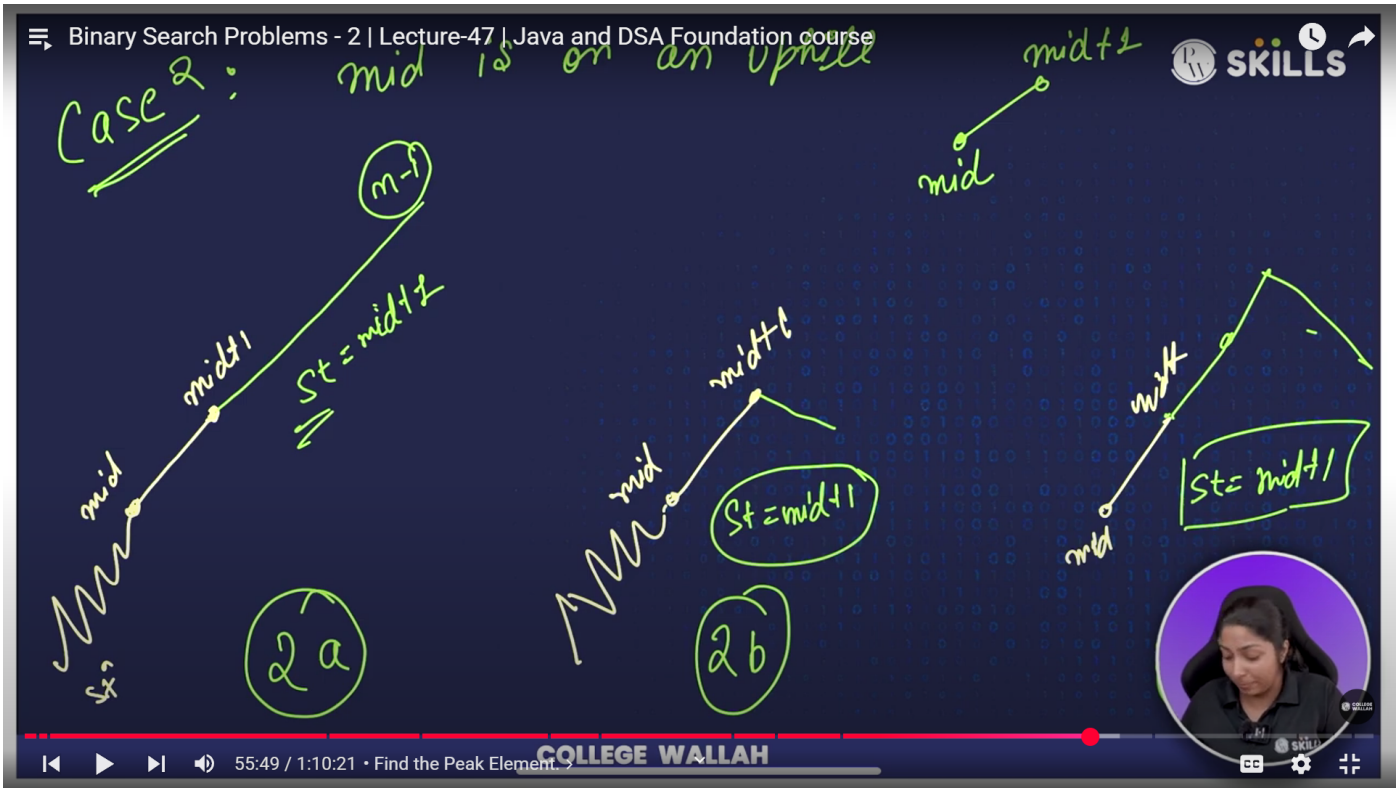


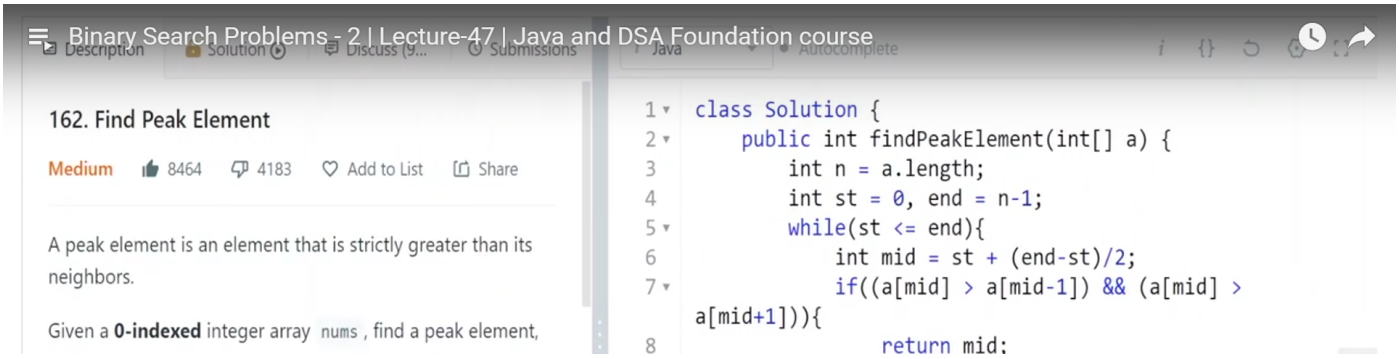

```java
public boolean search_(int[] a, int target) {
    int st = 0, end = a.length-1;
    while(st <= end){
        int mid = st + (end-st)/2;
        if(a[mid] == target) return true;
        if(a[st] == a[mid] && a[end] == a[mid]) {
            ++st;
            --end;
        }
        else if(a[mid] <= a[end]) {
            if(target > a[mid] && target <= a[end]){
                st = mid+1;
            } else {
                end = mid-1;
            }
        }
        else
        {
            if(target >= a[st] && target < a[mid]){
                end = mid-1;
            } else {
                st = mid+1;
            }
        }
    }
    return false;
}
```






Binary Search Problems - 2 | Lecture-47 | Java and DSA Foundation course
Case 1 : mid is peak
mid
mid-1
mid+1
if (a[mid] > a[mid-1] &&
a[mid] > a[mid+1])
return mid;
SKILLS
49:18 / 1:10:21 • Find the Peak Element
COLLEGE WALLAH


Binary Search Problems - 2 | Lecture-47 | Java and DSA Foundation course
Case 2 : mid is on an uphill
mid+1
mid
m-1
mid+1
st = mid+1
mid
st
2a
mid+1
mid
st = mid+1
2b
mid
st = mid+1
SKILLS
55:49 / 1:10:21 • Find the Peak Element
COLLEGE WALLAH


Binary Search Problems - 2 | Lecture-47 | Java and DSA Foundation course
Description
Solution
Discuss (9...
Submissions
Java
Autocomplete
162. Find Peak Element
Medium
8464
4183
Add to List
Share
A peak element is an element that is strictly greater than its neighbors.
Given a 0-indexed integer array nums , find a peak element,
class Solution {
public int findPeakElement(int[] a) {
int n = a.length;
int st = 0, end = n-1;
while(st <= end){
int mid = st + (end-st)/2;
if((a[mid] > a[mid-1]) && (a[mid] > a[mid+1])){
return mid;

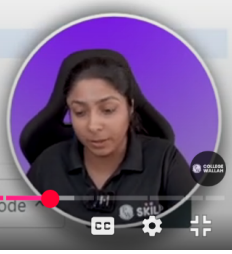
and return its index. If the array contains multiple peaks, return the index to **any of the peaks**.

You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in `O(log n)` time.

Example 1:

```
            }
            if(a[mid] < a[mid+1]){ // uphill/ascending
                st = mid+1;
            } else {
                end = mid-1;
            }
        }
        return -1;
    }
}
```



1:01:31 / 1:10:21 • Find the Peak Element. Code. >

Binary Search Problems - 2 | Lecture-47 | Java and DSA Foundation course

Explanation: Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.

Constraints:

- `1 <= nums.length <= 1000`
- -2^{31} `<= nums[i] <=` $2^{31} - 1$
- `nums[i] != nums[i + 1]` for all valid `i`.

Accepted 965.1K | Submissions 2.1M

Seen this question in a real interview before?

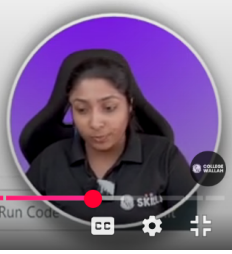
Yes | No

Companies

Related Topics

Similar Questions

```
class Solution {
    public int findPeakElement(int[] a) {
        int n = a.length;
        int st = 0, end = n-1;
        while(st <= end){
            int mid = st + (end-st)/2;
            if((mid == 0 || a[mid] > a[mid-1]) && (mid == n-1 || a[mid] > a[mid+1])){
                return mid;
            }
            if(a[mid] < a[mid+1]){ // uphill/ascending
                st = mid+1;
            } else {
                end = mid-1;
            }
        }
        return -1;
    }
}
```



1:03:49 / 1:10:21 • Find the Peak Element. Code. >

SKILLS

Problems on Binary Search

You have 'n'(n <= 10^5) boxes of chocolate. Each box contains a[i] (a[i] <= 10000) chocolates. You need to distribute these boxes among 'm' students such that the maximum number of chocolates allocated to a student is minimum.

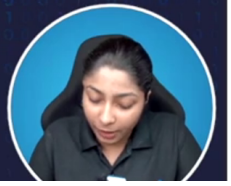
a) One box will be allocated to exactly one student.
b) All the boxes should be allocated.
c) Each student has to be allocated at least one box.
d) Allotment should be in contiguous order, for instance, a student cannot be allocated box 1 and box 3, skipping box 2.

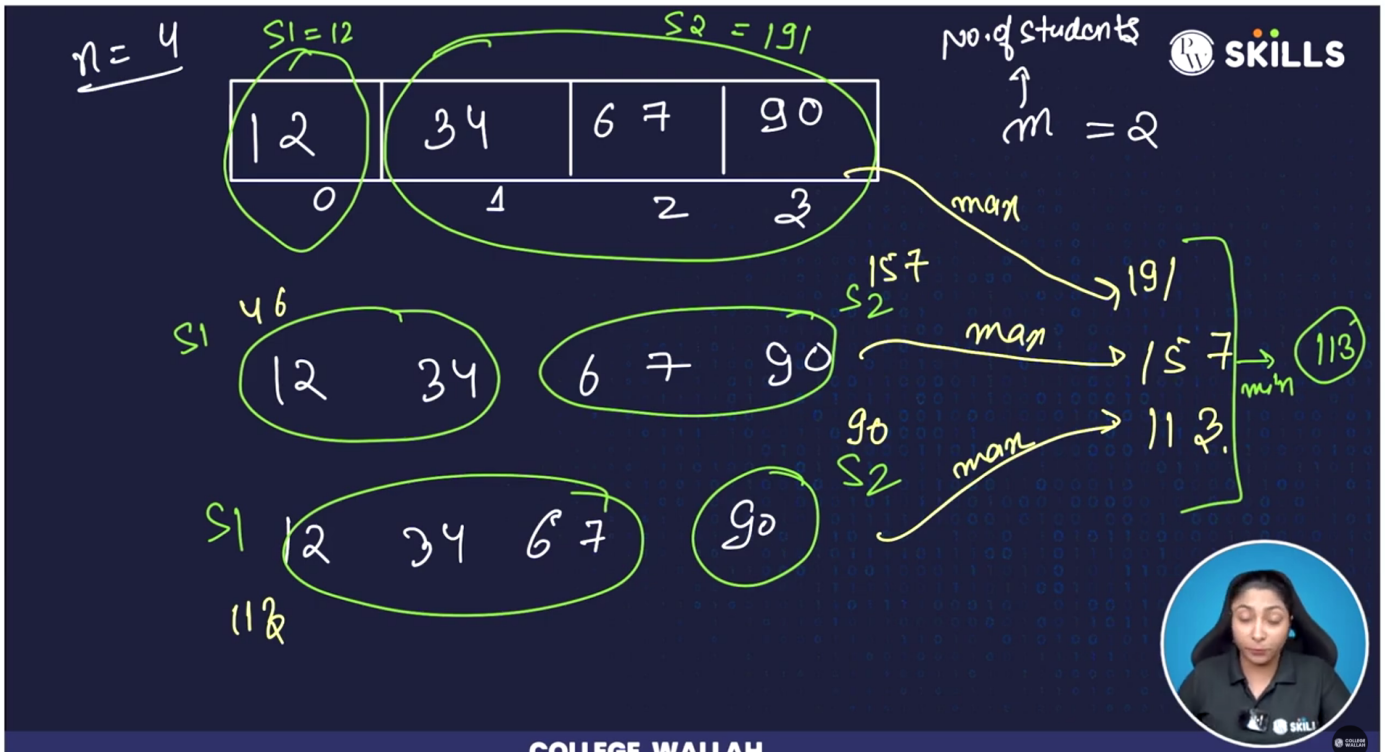
Calculate and return that minimum possible number.

Assume that it is always possible to distribute the chocolates.

The first line of input will contain the value of n, the number of boxes.
The second line of input will contain the n numbers denoting the number of chocolates in each box.
The third line will contain the m, number of students.

Input
4
12 34 67 90
2
Output
113




n = 4
S1 = 12
S2 = 191
No. of students
m = 2
SKILLS
12
34
67
90
0
1
2
3
max
191
S2
157
max
157
113
min
S1
46
12
34
67
90
90
S2
max
113
S1
12
34
67
90
112